On this page
Was this page helpful?
Yes No
Send feedback

*Last updated on **Nov 3, 2025***

# View and query the billing logs

This article explains how admins can monitor usage in their SAP Databricks accounts using the billing system tables.

## What are system tables?

System tables are an analytical store of your account's operational data found in the `system` catalog. System tables can be used for historical observability across your account. They are read-only and cannot be modified.

To grant access to the `billing` schema, a user that is both a metastore admin and an account admin must grant `USE` and `SELECT` permissions on the system schemas.

## Available billing tables

There are two tables you can use to monitor costs:

- `system.billing.usage`: Gives you access to account-wide billable usage data.
- `system.billing.list_prices`: Gives you access to a historical log of SKU pricing. A record gets added each time there is a change to a SKU price.

## Billable usage table schem

**Table path**: This system table is located at `system.billing.usage`.

The billable usage system table uses the following schema:

| Column name | Data type | Description | Example |
|---|---|---|---|
| `record_id` | string | Unique ID for this usage record | `11e22ba4-87b9-4cc2-9770-d10b894b7118` |
| `account_id` | string | ID of the account this report was generated for | `23e22ba4-87b9-4cc2-9770-d10b894b7118` |
| `workspace_id` | string | ID of the workspace this usage was associated with | `1234567890123456` |
| `sku_name` | string | Name of the SKU | `ENTERPRISE_SAP_ALL_PURPOSE_SERVERLESS_COMPUTE_EUROPE_FRANKI` |
| `cloud` | string | Cloud associated with this usage. Possible values are `AWS`, `AZURE`, and `GCP`. | `AWS` |
| `usage_start_time` | timestamp | The start time relevant to this usage record. Timezone information is recorded at the end of the value with `+00:00` representing UTC timezone. | `2025-01-09 10:00:00.000+00:00` |
| `usage_end_time` | timestamp | The end time relevant to this usage record. Timezone information is recorded at the end of the value with `+00:00` representing UTC timezone. | `2025-01-09 11:00:00.000+00:00` |
| `usage_date` | date | Date of the usage record, this field can be used for faster aggregation by date | `2025-01-01` |
| `custom_tags` | map | Custom tags propagated from the serverless budget policy associated with the usage | `{ "team": "marketing" }` |

| Column name | Data type | Description | Example |
|---|---|---|---|
| `usage_unit` | string | Unit this usage is measured in | `DBU` |
| `usage_quantity` | decimal | Number of units consumed for this record | `259.2958` |
| `usage_metadata` | struct | System-provided metadata about the usage, including IDs for compute resources and jobs (if applicable). See Usage Metadata. | See Usage metadata |
| `identity_metadata` | struct | System-provided metadata about the identities involved in the usage. See Identity Metadata. | See Identity metadata |
| `record_type` | string | Whether the record is original, a retraction, or a restatement. The value is `ORIGINAL` unless the record is related to a correction. | `ORIGINAL` |
| `ingestion_date` | date | Date the record was ingested into the `usage` table | `2024-01-01` |
| `billing_origin_product` | string | The product that originated the usage. Some products can be billed as different SKUs. For possible values, see Product. | `JOBS` |
| `product_features` | struct | Details about the specific product features used. | `is_serverless: true` |
| `usage_type` | string | The type of usage attributed to the product or workload for billing purposes. Possible values are `COMPUTE_TIME`, `STORAGE_SPACE`, `NETWORK_BYTES`, `NETWORK_HOUR`, `API_OPERATION`, or `TOKEN`. | `STORAGE_SPACE` |

## Pricing table schema

**Table path**: This system table is located at `system.billing.list_prices`.

The pricing system table uses the following schema:

| Column name | Data type | Description | Example |
|---|---|---|---|
| `price_start_time` | timestamp | The time this price became effective in UTC | `2023-01-01T09:59:59.999Z` |
| `price_end_time` | timestamp | The time this price stopped | `2023-01-01T09:59:59.999Z` |

| Column name | Data type | Description | Example |
|---|---|---|---|
| | | being effective in UTC | |
| `account_id` | string | ID of the account this report was generated for | `1234567890123456` |
| `sku_name` | string | Name of the SKU | `ENTERPRISE_SAP_ALL_PURPOSE_SERVERLESS_COMPUTE_EUROPE_FRANKFURT` |
| `cloud` | string | Name of the Cloud this price is applicable to. Possible values are `AWS`, `AZURE`, and `GCP`. | `AWS`, `AZURE`, or `GCP` |
| `currency_code` | string | The currency this price is expressed in. The value will be `CU` | `CU` |
| `usage_unit` | string | The unit of measurement that is monetized. Possible values are `DBU`, `DSU`, `HOUR`, and `GB`. | `DBU` |
| `pricing` | struct | A structured data field that includes pricing info at the published list price rate. The key `default` will always return a single price that can be used for simple long-term estimates. The key `promotional` represents a temporary promotional price that all customers get which could be used for cost estimation during the temporary period. The key `effective_list` resolves list and promotional price, and contains the effective list price used for calculating the cost. Some pricing models might also include additional keys that provide more detail. | `{`<br>`"default": "0.10",`<br>`"promotional": {"default": "0.07"},`<br>`"effective_list": {"default": "0.07"}`<br>`}` |

## How to read the usage table

Every billing record in the `usage` table includes columns that attribute the usage amount to the specific resources, identities, and products involved. To help attribute usage to specific workloads, features, and identities, use the following columns:

- The `usage_metadata` column includes a struct with information about the resources or objects involved in the usage.
- The `identity_metadata` column includes information about the user or service principal that incurred the usage.
- The `custom_tags` column includes tags added by serverless budget policies. For more information, see Attribute usage with serverless budget policies.
- The `billing_origin_product` column give you information about the exact product used.

### Usage metadata reference

The values in `usage_metadata` are all strings that tell you about the workspace objects involved in the usage record.

Only a subset of these values is populated in any given usage record, depending on the compute type and features used. The following table outlines the values you can expect to populate in your SAP Databricks account.

| Value | Description |
|---|---|
| `job_id` | ID of the job associated with the usage record |

| Value | Description |
| --- | --- |
| `warehouse_id` | ID of the SQL warehouse associated with the usage record |
| `job_run_id` | ID of the job run associated with the usage record |
| `notebook_id` | ID of the notebook associated with the usage |
| `endpoint_name` | The name of the model serving endpoint or vector search endpoint associated with the usage record |
| `endpoint_id` | ID of the model serving endpoint or vector search endpoint associated with the usage record |
| `run_name` | Unique user-facing name of the Foundation Model Fine-tuning run associated with the usage record |
| `job_name` | User-given name of the job associated with the usage record |
| `notebook_path` | Workspace storage path of the notebook associated with the usage |
| `source_region` | Region of the workspace associated with the usage. Only returns a value for serverless networking-related usage. |
| `destination_region` | Region of the resource being accessed. Only returns a value for serverless networking-related usage. |
| `budget_policy_id` | ID of the serverless budget policy attached to the workload |

## Identity metadata reference

The `identity_metadata` column provides more information about the identities involved in the usage.

- The `run_as` field logs who ran the workload. This values is only populated for certain workload types listed in the table below.
- The `owned_by` field only applies to SQL warehouse usage and logs the user or service principal who owns the SQL warehouse responsible for the usage.

### run_as identities

The identity recorded in `identity_metadata.run_as` depends on the product associated with the usage. Reference the following table for the `identity_metadata.run_as` behavior:

| Workload type | Identity of `run_as` |
| --- | --- |
| Serverless compute for jobs | The user or service principal defined in the `run_as` setting. By default, jobs run as the identity of the job owner, but admins can change this to be another user or service principal. |
| Serverless compute for notebooks | The user who ran the notebook commands (specifically, the user who created the notebook session). For shared notebooks, this includes usage by other users sharing the same notebook session. |
| Foundation Model Fine-tuning | The user or service principal that initiated the fine-tuning training run. |
| Data profiling | The user who created the profile. |

## Billing origin product reference

The `billing_origin_product` column shows the Databricks product associated with the usage record. The values include:

- `JOBS`
- `SQL`
- `MODEL_SERVING`
- `INTERACTIVE`
- `DEFAULT_STORAGE`
- `VECTOR_SEARCH`
- `DATA_QUALITY_MONITORING`
- `FOUNDATION_MODEL_TRAINING`
- `AGENT_EVALUATION`
- `NETWORKING`: Costs associated with connecting serverless compute to your resources

# Sample queries

Use the following queries to gain insights into usage in your account:

- Join the list_prices table with the usage table
- Identify expensive notebooks
- Identify expensive jobs
- Report on DBUs consumed by a particular user
- Report on DBUs consumed by workloads that share a custom tag

## Join the list_prices table with the usage table

The `list_prices` table includes list prices over time for each available SKU. You can join the `usage` table to view the listing cost of certain usage.

For example, the following query returns the total cost attributed to a particular user during a month. Be sure to insert a valid identity in the bracketed section.

SQL

```
SELECT
 SUM(usage.usage_quantity * list_prices.pricing.effective_list.default)
 as `Total Dollar Cost`
FROM system.billing.usage
JOIN system.billing.list_prices ON list_prices.sku_name = usage.sku_name
WHERE identity_metadata.run_as = {{'sample identity'}}
AND usage.usage_end_time >= list_prices.price_start_time
AND (list_prices.price_end_time IS NULL OR usage.usage_end_time < list_prices.price_end_time)
AND usage.usage_date BETWEEN "2025-04-01" AND "2025-04-15"
```

## Identify expensive notebooks

This query returns a list of notebooks and how many DBUs each notebook consumed, in descending order by DBU consumption:

SQL

```
SELECT
  usage_metadata.notebook_id,
  usage_metadata. notebook_path,
  SUM(usage_quantity) as total_dbu
FROM
  system.billing.usage
WHERE
  usage_metadata.notebook_id is not null
  and billing_origin_product = 'INTERACTIVE'
  and product_features.is_serverless
  and usage_unit = 'DBU'
  and usage_date >= DATEADD(day, -30, current_date)
GROUP BY
  1,2
ORDER BY
  total_dbu DESC
```

## Identify expensive jobs

This query returns a list of jobs and how many DBUs each job consumed, in descending order by DBU consumption:

SQL

```
SELECT
  usage_metadata.job_id,
  usage_metadata.job_name,
  SUM(usage_quantity) as total_dbu
FROM
  system.billing.usage
WHERE
  usage_metadata.job_id is not null
  and usage_unit = 'DBU'
  and usage_date >= DATEADD(day, -30, current_date)
  and sku_name like '%JOBS_SERVERLESS_COMPUTE%'
GROUP BY
  1,2
ORDER BY
  total_dbu DESC
```

## Report on DBUs consumed by a particular user

This query returns a list of notebooks and jobs run by a particular user or service principal, and the number of DBUs consumed by each workload. Be sure to insert a valid user email address using the parameters.

SQL

```
SELECT
  usage_metadata.job_id,
  usage_metadata.job_name,
  usage_metadata.notebook_id,
  usage_metadata. notebook_path,
  SUM(usage_quantity) as total_dbu
FROM
  system.billing.usage
WHERE
  identity_metadata.run_as = format_string('%s@%s.%s', :user_name, :domain_name, :top_level_domain)
  and usage_date >= DATEADD(day, -30, current_date)
GROUP BY
  1,2,3,4
ORDER BY
  total_dbu DESC
```

## Report on DBUs consumed by workloads that share a custom tag

This query returns a list of jobs that share the same custom tag, and the number of DBUs consumed by each workload:

SQL

```
SELECT
  usage_metadata.job_id,
  usage_metadata.job_name,
  usage_metadata.notebook_id,
  usage_metadata. notebook_path,
  SUM(usage_quantity) as total_dbu
FROM
  system.billing.usage
WHERE
  custom_tags.<key> = '<value>'
  and billing_origin_product in ('JOBS','INTERACTIVE')
  and product_features.is_serverless
  and usage_unit = 'DBU'
  and usage_date >= DATEADD(day, -30, current_date)
GROUP BY
  1,2,3,4
ORDER BY
  total_dbu DESC
```